

Introducción

Un sistema de computación está formado por múltiples componentes de *hardware* y *software*. La única manera de lidiar con la complejidad de estos sistemas es *modularizar*, es decir, crear pequeños componentes interconectados para construir sistemas más complejos. La siguiente figura muestra una estructura típica de componentes en diferentes capas de abstracción un sistema de computación actual de propósitos generales.

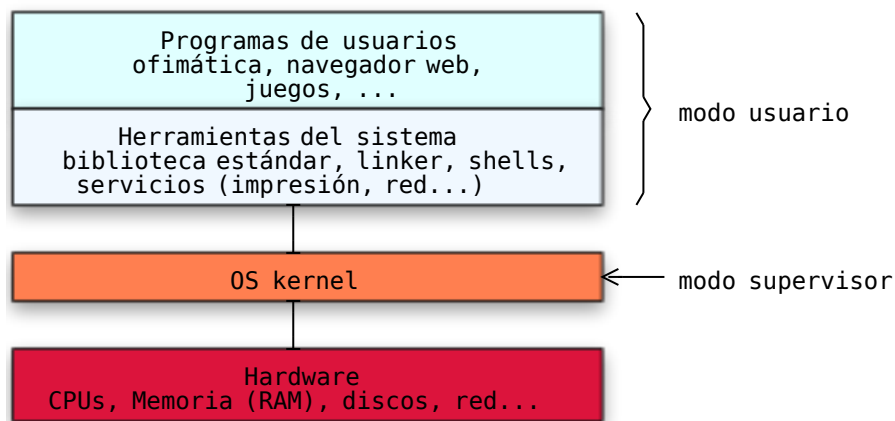


Figura 1: *Capas de un sistema de computación.*

Para comprender mejor el rol que cumple un *sistema operativo* es conveniente analizar la historia del desarrollo y la evolución de sistemas de computación.

Los primeros sistemas de cómputo no tenían *interfaces* para el usuario/ programador. Los códigos de las instrucciones y los valores de los datos de un programa se debían introducir manualmente en la memoria principal del sistema por medio de cableados y conmutadores.

Obviamente esto era una tarea sumamente tediosa y propensa a errores y sólo era posible hacer pequeños programas para las pequeñas memorias de aquellos días.

A medida que la capacidad de memoria crecía, se desarrollaron dispositivos de almacenamiento para los programas y datos de entrada y salida:

1. Tarjetas y cintas de papel perforadas
2. Medios magnéticos como las cintas, cilindros y discos
3. Actualmente: Memorias no volátiles de estado sólido

Para el caso de las cintas y tarjetas perforadas de papel, se desarrollaron dispositivos que comúnmente eran computadoras de propósitos específicos o *teletipos* (*tty*) adaptados que permitían que los programadores *escriban* el programa en la *teletipo* (una máquina de escribir electrónica) y ésta producía las cintas o tarjetas perforadas. La computadora incluía un *lector* de tarjetas o cintas que introducía las instrucciones y datos de los programas en la memoria y un *impresor* que perforaba en las tarjetas o cintas los datos de salida.

En esos días (principios de 1950) no existía el concepto de sistema operativo. Cada programa debía ser *auto-contenido*, o sea que debía contener todas las instrucciones necesarias aún para las operaciones rutinarias de entrada-salida, por lo que cada programador debía conocer todos los detalles del sistema.

Actualmente podemos encontrar sistemas **sin SO** comúnmente en pequeños **micro-controladores** o *digital signal processors (DSPs)* en donde la única aplicación que ejecutan es un programa monolítico que contiene todas las funciones necesarias. Estos sistemas implementan un mini sistema de cómputo de propósitos específicos.

Algunos SOs para pequeños procesadores tienen la forma de una *biblioteca* que se enlaza con el código de la aplicación. Un ejemplo es el *Real-Time Operating Systems (RTOS)* usados para el desarrollo de *embedded systems*. Esto es similar a los primeros sistemas operativos de la década de 1950.

Los primeros sistemas operativos

Obviamente, los programadores debían escribir mucho código que podría ser *reusable* por todos. En particular las operaciones de entrada-salida y otras funciones de uso común o *rutinario* (de ahí el nombre *routines*) como funciones matemáticas y otras.

Los **sistemas** denominados **por lotes** o **batch systems** permitían cargar programas y **bibliotecas** (contenedores de rutinas). Los programadores depositaban sus programas y datos de entrada(*jobs*). Luego un operador los agrupaba (en lotes) y los ordenaba en una cola para que el sistema los cargara y ejecutara en secuencia. Cada lote era precedido por una secuencia de comandos que especificaba las *bibliotecas* requeridas. El sistema operativo (SO) las cargaba en memoria si no habían sido cargadas previamente.

En estos tipos de sistemas cada programa tenía acceso a todos los recursos (CPU, memoria, dispositivos de entrada-salida, etc) del sistema.

Sistemas operativos multitarea

Las operaciones de entrada-salida dejan demasiado tiempo a la CPU ociosa, la cual sólo debía esperar (en un ciclo, verificando algún registro del controlador del dispositivo) a que esas operaciones finalicen.

Con el objetivo de maximizar el uso de CPU, los próximos sistemas operativos evolucionaron para cargar en memoria varios programas (*procesos*) y ejecutarlos concurrentemente asignando la CPU a otra tarea cuando la anterior iniciaba una operación de entrada-salida. Estos sistemas se denominan **sistemas de multiprogramación** o **multitareas**.

De esta manera se logra que se utilice la cpu y las operaciones de E/S en paralelo.

Generalmente los controladores de dispositivos generan interrupciones al ocurrir un evento como la finalización de una operación de lectura o escritura de un bloque en el disco.

Ante una interrupción recibida por la CPU, ésta hace que el flujo de ejecución *salte* a una *rutina de atención* o *trap handler*. De ese modo, el kernel retoma el control.

En estos sistemas, las tareas *cooperan* cediendo la CPU en cada llamada al sistema por lo que se conocen como *SO multitarea cooperativa*. Las primeras versiones de MS-Windows tenían esta característica.

Contextos de ejecución

Cuando la CPU está ejecutando código de un proceso, está operando en *modo usuario*. Al ocurrir una interrupción, la CPU *salta* a ejecutar el *interrupt handler* en *modo kernel*.

Dentro del código del kernel hay que identificar si el código se está ejecutando en el *contexto de un proceso*, es decir que una interrupción ocurrió mientras la CPU estaba en modo usuario o no (el que fue interrumpido fue código del kernel). En éste último caso, el el kernel, no existe la noción de *proceso corriente*.

Cambios de contexto

Cuando el kernel decide quitar la CPU a un proceso y asignarla a otro se realiza un *context switch* y consiste en los siguientes pasos:

1. Guardar (salvar) el estado (*registros de la CPU*) en el descriptor del proceso (comúnmente llamado *contexto*).
2. Recuperar en los registros de la CPU el contexto guardado del próximo proceso.

Esto producirá un *salto* de un punto de ejecución de un hilo de ejecución al otro ya que cambian el *program counter (pc)* y el *stack pointer (sp)*.

En el capítulo de gestión de procesos se describe este mecanismo en mayor detalle.

Protección

Estos sistemas requieren de algún mecanismo de *protección*:

- **Confinamiento:** Cada proceso debe acceder *sólo a su propio espacio de memoria*.
- **Control:** Los procesos no deberían poder ejecutar ciertas *instrucciones privilegiadas*, como deshabilitar interrupciones o usar registros especiales que permitan quitarle el control al kernel.

El kernel es un binario independiente y auto-contenido. Comúnmente *tiene acceso a todos los recursos del sistema*.

Cada aplicación en ejecución (*proceso*) tendrá su *propio espacio de direcciones de memoria*.

La protección de memoria requiere de un mecanismo especial para que una tarea requiera un servicio del SO ya que no puede simplemente hacer una llamada a una función del kernel (ya que son programas diferentes).

Es necesario que el hardware disponga de un mecanismo para acceder a los servicios del SO como por ejemplo, una instrucción *trap* (trampa).

Una *Llamada al sistema (syscall)* es una función que permite a los procesos solicitar un servicio o recurso al kernel del SO. La *interfaz de un OS* comúnmente se describe como el conjunto de *syscalls* que ofrece a las aplicaciones.

En una *syscall* el flujo de ejecución del proceso se interrumpe *saltando* del espacio de memoria de un proceso al espacio del kernel. Luego, el kernel procesa el *syscall* y retorna al espacio de usuario del proceso el cual continúa su ejecución.

Durante el procesamiento de una *syscall* el kernel puede decidir quitarle la CPU al proceso corriente (porque ya) y continuar la ejecución de otro.

Generalmente, en un salto al kernel, la CPU cambia su *nivel de privilegio* o *modo de operación*. La mayoría de las CPU modernas incluyen al menos dos modos de ejecución: *modo kernel (o supervisor)* y *modo usuario*.

La CPU ejecuta el código del núcleo del SO en *modo kernel* o *supervisor* que es más privilegiado que el modo usuario en el cual ejecutan los *procesos de usuario*. En modo kernel la CPU permite la ejecución de *instrucciones privilegiadas* y acceder a registros de dispositivos, habilitar y deshabilitar interrupciones, entre otras. Estas instrucciones privilegiadas no son accesibles en *modo usuario*.

Sistemas de tiempo compartido

A medida que se incorporaron dispositivos interactivos como las *consolas* (teclado + pantalla), se comenzaron a desarrollar aplicaciones *interactivas*, las cuales permitían a usuarios a ingresar datos y visualizar resultados en una *sesión* de ejecución de un programa.

En un sistema con multi-programación una tarea puede monopolizar la CPU o hacer que el sistema sea muy poco interactivo ante usuarios (éstos podrían tener demoras significativas si la tarea en ejecución no hace una llamada al sistema).

Para evitar este problema el kernel puede *quitarle (preempt) la CPU* a una tarea que ya la ha usado por un intervalo de tiempo determinado y asignársela a otra tarea y garantizar un progreso más uniforme. Esto se logra mediante el manejo de las *interrupciones periódicas de un reloj*, las cuales son *atrapadas* por el *kernel* y actuando en consecuencia.

Los sistemas *multi-usuarios* soportan diferentes espacios de trabajo para diferentes usuarios (*home directories*) con sistemas de control de acceso a los recursos. Muchos de estos sistemas permiten la conexión de múltiples *terminales* o *consolas* a la computadora la cual se usa de manera compartida.

Actualmente las computadoras personales permiten *múltiples consolas virtuales*. Una *consola* o *terminal* virtual es un programa que *emula* una terminal física (hardware). También existen aplicaciones que *emulan* terminales remotas por red como telnet, ssh y aplicaciones de *remote desktop*.

Objetivos de un sistema operativo

Un SO tiene varios objetivos. El principal es ofrecer una *máquina abstracta* que oculta los detalles del hardware.

Uno de los principales objetivos es *virtualizar* recursos de hardware.

Esta *virtualización* permite su mejor utilización ya que incluye el *multiplexado* de los mismos.

Básicamente un SO crea *abstracciones* de los recursos.

Por ejemplo, un *proceso* representa una unidad de ejecución en una CPU (virtual) y su memoria. Un SO multitarea *multiplexa* la CPU entre los procesos.

En algunos casos como en la gestión de la memoria, ésta se *divide* lógica o físicamente para asignarle espacios a cada proceso y al kernel. Un sistema de memoria virtual permite ejecutar un conjunto de procesos cuya demanda de memoria es mayor que la memoria física correspondiente mediante la técnica de *swapping* (que se describe en el capítulo [memory.md]).

Otro ejemplo de *abstracción* del hardware es el **sistema de archivos** el cual ofrece a los procesos una *interfaz* de operaciones sobre *archivos* y *directorios* (*carpetas*), ocultando los detalles técnicos de los dispositivos de almacenamiento (discos, cintas, etc).

El diseño de UNIX abstrae la interacción con los dispositivos de E/S presentando a éstos como *archivos especiales* que se pueden leer y/o escribir mediante la conocida API de llamadas al sistema sobre archivos como `open(...)` , `read(...)` , `write(...)` , `close(...)` y otras.

Esta *virtualización* o *multiplexado* hace que el SO también sea un **administrador de recursos** ya que debe incluir algoritmos de planificación de la asignación de los recursos para optimizar su uso.

Hardware

El siguiente diagrama muestra un esquema simplificado de una arquitectura general del hardware.

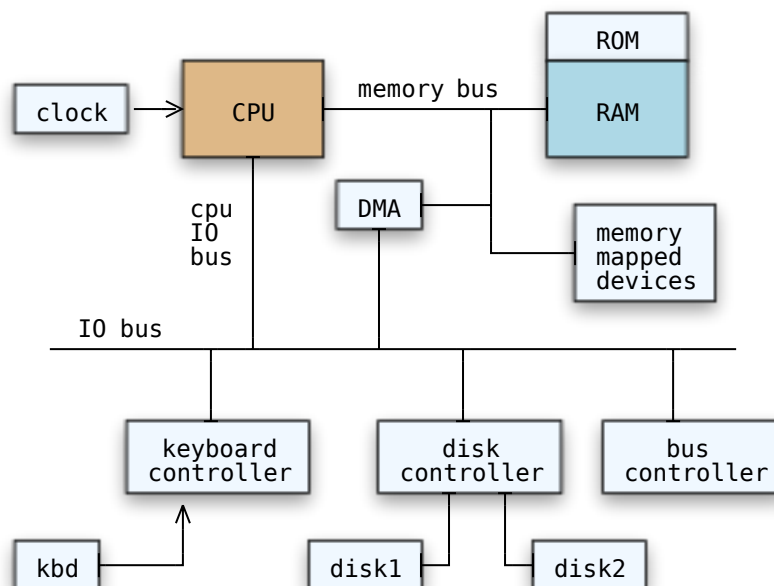


Figura 2: Estructura típica de hardware.

Como se puede apreciar en la figura de arriba, el hardware de un sistema de computación consta de diferentes componentes.

- *CPU*: Uno o más procesadores.

- *ROM*: Memoria *no volátil* que contiene código de arranque (boot) y datos de la plataforma de hardware.
- *RAM*: Memoria principal (*random-access-memory*). Esta memoria es *volátil*.
- *I/O bus and ports*: Bus de I/O al cual se conectan los dispositivos de entrada/salida como en la plataforma x86 de Intel.
- *Device controller*: expone un conjunto de *puertos* o *registros* de control y estado para un cierto tipo de dispositivos. Un *controller* puede manejar varios dispositivos del mismo tipo. Existen varios estándares como [IDE/PATA](#) para dispositivos de almacenamiento, [USB](#) para varios tipos de dispositivos.
- *Memory mapped devices (MMIO)*: Dispositivos de entrada salida conectados al *memory bus*. Ejemplos: *display/video controllers* y otros.
- *Direct-Memory-Access (DMA)*: Dispositivo que inter-conecta controladores de dispositivos al bus de memoria para permitir la transferencia de datos desde/hacia los dispositivos sin necesidad que intervenga la CPU.

Un SO debe conocer los detalles de estos componentes. Los módulos de software que interactúan con los controladores de dispositivos se denominan *device drivers*.

Diseño

Los SO están diseñados en forma modular, en subsistemas:

1. Núcleo (kernel):

- Gestión de *procesos* y usuarios
- Gestión de la *memoria*
- Subsistema de *entrada-salida*: Conjunto de *device drivers*
- Sistemas de *archivos*
- Subsistema de *red* (comunicaciones)
- Mecanismos de *Seguridad*: Generalmente es un sistema transversal a los demás subsistemas

2. Aplicaciones y *bibliotecas* del sistema (ej: `libc` en sistemas tipo UNIX)

- *Utilidades*: Shells, comandos básicos, editores de texto, ...
- *Servicios*: Procesos de usuario (*daemons*) que brindan servicios de uso común como el servicio de impresión u otros.

Algunos de las procesos de usuario que forman parte de las utilidades y servicios del sistema ejecutan con mayores *privilegios* que los programas de usuario comunes.

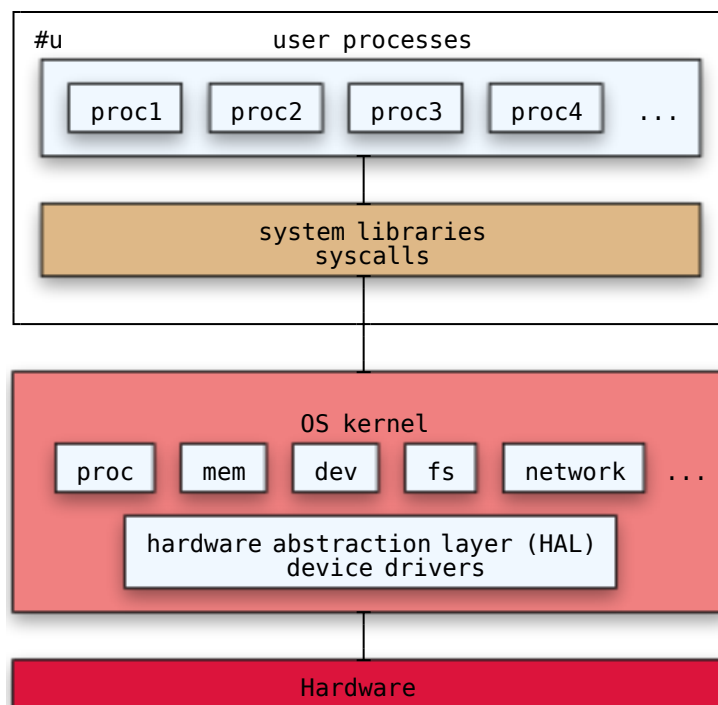


Figura 3: Arquitectura de un sistema de computación.

En la figura 3, los programas de usuario utilizan (se enlazan con) las bibliotecas del sistema y otras de propósitos específicas. La biblioteca del sistema contiene la *interfaz del programador de aplicaciones (API)* la cual incluye las funciones que implementan las *llamadas al sistema* y otras utilidades de uso rutinario.

Una *llamada al sistema (syscall)* activa el mecanismo de interrupciones para saltar al código del kernel que la procesa.

El subsistema de E/S, representado como *dev* en la figura 2, incluye los *device drivers* los cuales son módulos de software que *controlan* una cierta clase de dispositivos, como discos IDE, teclado, mouse, pantalla, etc.

Uno de los desafíos en el diseño de un SO es encapsular el software dependiente de la plataforma de hardware del resto del código independiente de la plataforma. Esto se logra definiendo abstracciones de los componentes de hardware para facilitar su portabilidad a otras arquitecturas.

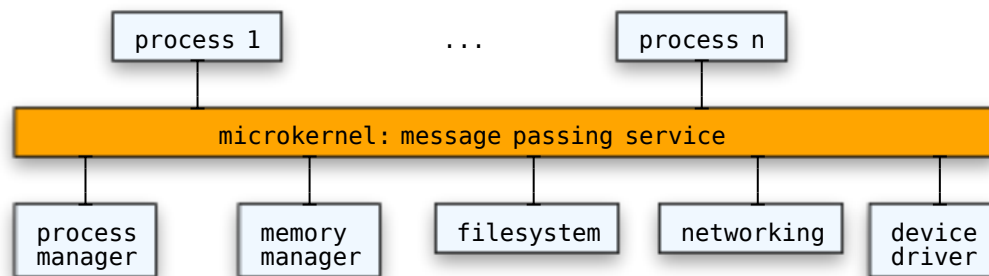
Esta capa de abstracción en el *kernel* se conoce como *hardware abstraction layer (HAL)* y se considera la capa más baja dentro del kernel.

Los diseños de los SO se puede clasificar en:

- **Monolíticos:** Los subsistemas del *kernel* se *enlazan* en un solo *ejecutable*. Cada módulo puede *invocar* a funciones de otros módulos.
 - Ventajas: Simple y eficiente.
 - Desventajas: Un error en un módulo puede afectar a todo el núcleo.

Ejemplos: UNIX, [GNU-Linux][<https://www.gnu.org/gnu/linux-and-gnu.en.html>], [FreeBSD](#) y [Fuchsia](#).

- **Microkernels:** Cada subsistema del *kernel* se compila como un *ejecutable* independiente. Para *invocar* un servicio de otro subsistema se usa un mecanismo de comunicación basado en *mensajes*.



- Ventajas: Un error (ejemplo, de memoria) en un subsistema no afecta a los demás ya que cada componente es independiente y está confinado en su propia área de memoria. Fácilmente se puede convertir en un SO *distribuido* (subsistemas ejecutándose en diferentes nodos de una red).
- Desventajas: El mecanismo de pasaje de mensajes crea una *sobrecarga* computacional importante afectando su rendimiento.

Ejemplos: [Minix](#), [L4](#), [MS-Windows](#)

- **Máquinas virtuales:** El sistema tiene un *hypervisor* que se encarga de *multiplexar* el hardware a los SO *huéspedes* que se ejecutan encima (en modo usuario). El *hypervisor* ofrece a cada SO huésped un *ambiente aislado* de ejecución.

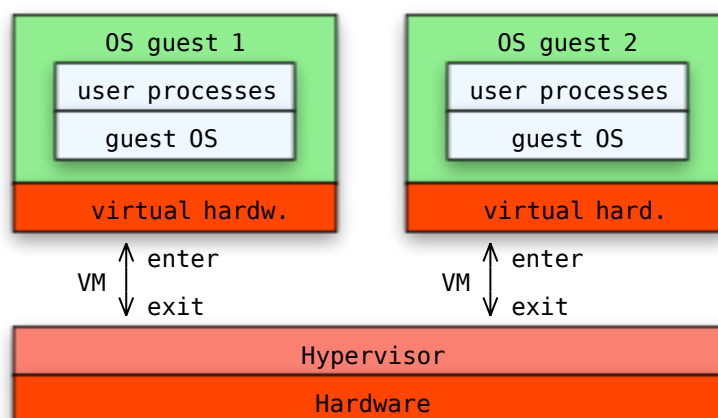


Figura 4: Esquema de un SO basado en máquinas virtuales.

Cuando el *hypervisor* está integrado en un SO, a éste se le denomina el *host kernel* (*anfitrión*) como por ejemplo [Linux KVM](#) que provee mecanismos básicos para las *transiciones* entre un SO *huésped* (*guest*) y el *host* y redirigir a un virtualizador en modo usuario como [gemu](#) o [VirtualBox](#).

Generalmente para poder implementar un hypervisor, el hardware debe proveer mecanismos como nuevos *modos de operación* de la CPU como por ejemplo *Intel VMX root mode* (modo del *anfitrión*) y *VMX non-root-mode* (modo del *huésped*) y otras estructuras de datos como *registros sombras* (*shadowed*).

Algunas arquitecturas modernas como RISC-V incluyen modos de protección o *privilegios* para facilitar la implementación de la virtualización.

Ejemplos: [Xen](#) e IBM System 370.

! No se debe confundir la virtualización con *contenedores*. Un contenedor es un *grupo de procesos* y recursos (memoria, filesystem, dispositivos y otros) que son independientes del resto del sistema. Esto permite la creación de *sistemas auto-contenidos minimalistas*, es decir que incluyen sólo las dependencias requeridas. Se ejecutan en forma *confinada o aislada* del resto del sistema. El SO debe proveer mecanismos para su ejecución. Ejemplos: GNU-Linux *cgroups* y FreeBSD *jails*. Aplicaciones como [Docker](#) permiten crear, configurar y ejecutar contenedores.

Consideraciones de diseño

Muchos SO actuales tienen una arquitectura híbrida, generalmente con un kernel pequeño monolítico y otros componentes *fuera de él* comunicándose por mensajes. Ejemplos de estos diseños son MS-Windows, Mac-OS y FreeBSD.

Algunos SO modernos, aún cuando sean monolíticos como GNU-Linux, son altamente modulares, permitiendo el desarrollo de ciertos subsistemas como *device drivers* o *filesystems* como *módulos de carga y descarga dinámica* (ver [Linux Kernel Modules](#)), permitiendo al administrador del sistema cargar o descargar componentes *en caliente*, es decir sin necesidad de reiniciar (*reboot*) el sistema.

Un principio de diseño sugerido para los SO es definir e implementar *mecanismos* en las capas mas bajas y que las *políticas* se definan en los niveles mas altos. Esto garantiza una buena *adaptabilidad* de un kernel a diferentes escenarios.

Interfaz de un SO

Un SO presenta a los desarrolladores de aplicaciones interfaces, tanto a nivel de código fuente como en formatos binarios.

Application Programming Interface (API): Interfaz a nivel de *código fuente* que incluye las llamadas al sistema y las funciones (y tipos de datos) de la biblioteca estándar.

El estándar **portable OS interface (POSIX)** define una *API* de portabilidad a nivel de sistema y usuario como también con un conjunto de *utilidades* (como *shells* y otros comandos) con el fin de compatibilizar variantes del SO *UNIX*.

En particular, éste estándar define la *API* como declaraciones de funciones y tipos de datos en `C`.

Application Binary Interface (ABI): Interfaz de bajo nivel que especifica los formatos binarios y convenciones para permitir la interacción de programas de usuario con las *bibliotecas* y con el *kernel* del sistema operativo.

Los sistemas operativos generalmente especifican una *ABI* para cada plataforma de hardware (x86, x86-64, ARM, risc-v, etc). La *ABI* incluye:

1. Convenciones para las *invocaciones a funciones*.
2. Convenciones para los *syscalls*.
3. Formato de archivos binarios (*objetos y ejecutables*)

A modo de ejemplo, GNU-Linux usa binarios en formato **Executable and Linking Format (ELF)** para todas las plataformas soportadas. La convención para las llamadas a funciones en x86 (32 bits) es que los parámetros se *apilan* por el invocante en orden reverso (de derecha a izquierda) y en x86-64 se pasan hasta 6 argumentos en los registros `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` (el resto en la pila en orden reverso). Los valores retornados quedan en el registro `eax/rax`, respectivamente. En ambas arquitecturas se usa las instrucciones `call/system` para una invocación.

En arquitecturas RISC, como RISC-V, los parámetros se pasan en los registros `a0`, ..., `a7` y el valor retornado queda en `a0`.

Estas convenciones deben ser respetadas por cualquier compilador de la plataforma.

Usuarios, sesiones, shell y variables de ambiente

Generalmente el uso de un sistema multi-usuario se basa en *sesiones*. Al comienzo un usuario debe presentar sus *credenciales* como su identificador y una *contraseña*. Luego el sistema lanza un *shell*, el cual puede basarse en una interfaz de texto (*consola*) o gráfica. Luego el usuario puede *interactuar* con el sistema lanzando otros programas para finalmente cerrar la sesión.

El *shell* define un conjunto de *variables de ambiente* que el usuario y los programas pueden usar y modificar.

Algunas variables de ambiente comunes en sistemas tipo UNIX son:

- `HOME` : Directorio o espacio de trabajo del usuario.
- `PATH` : Lista de directorios de búsqueda de programas.
- `PWD` : Directorio corriente (*print working directory*).
- `SHELL` : Programa (path) shell.
- `LANGUAGE` : Lenguaje de la interfaz de usuario

Se puede usar el comando `echo` para acceder al valor de una variable de ambiente. El valor de una variable `V` se accede con la expresión `$V` en UNIX (`%V` en MS-Windows).

Es posible listar las variables de ambiente con el comando `printenv` en UNIX o `set` en MS-Windows. Una variable de ambiente se puede crear o modificar su valor mediante el comando `var=value` o `export` (variable global), dependiendo del *shell*. En MS-Windows se puede crear/modificar una variable con el comando `set` .

El siguiente ejemplo muestra parte de la *API* que nos ofrece un SO *POSIX*.

c

```
#include <sys/types.h> /* system data types */
#include <unistd.h>     /* UNIX standard API */
#include <stdio.h>      /* I/O standard API */

int main(int argc, char *argv[], char *envv[])
{
    int i;
    printf("Process id: %d\n", getpid()); /* getpid() is a syscall */
    printf("Command line arguments:\n");
    for (i = 0; i < argc; i++)
        printf("%s\n", argv[i]);
    printf("Session environment variables:\n");
    for (i = 0; envv[i]; i++)
        printf("%s\n", envv[i]);
    exit(0); /* exit syscall: process exit code */
}
```

En este ejemplo se puede observar que:

1. Un programa de usuario comienza por una función denominada `main` que recibe como parámetros el número de los argumentos de la línea de comandos, un arreglo de punteros (strings) a ellos y un arreglo (null-terminated) de strings con las variables de ambiente de la forma `variable=valor` .

2. Los programas interactúan con los dispositivos de entrada/salida (en este caso la *consola* o *terminal*) indirectamente por medio de funciones de biblioteca, las cuales a su vez ejecutan *syscalls*. En este caso `printf()` , internamente ejecuta el *syscall* `write(1, buffer, count)` . El *descriptor de archivo* cero (0) está conectado a la entrada estándar (generalmente el teclado). Los descriptores 1 y 2 refieren a la *salida* y *salida de errores* estándar, asociados a la pantalla.
3. Las llamadas al sistema (como `getpid()` o `exit(n)`) están *implementadas* como funciones de la *biblioteca estándar*.
4. Al compilar este programa (con `gcc -o myprog myprog.c`) el *linker* lo *enlaza* con la biblioteca estándar.

Inicio del sistema (booting)

Al encenderse el sistema éste está configurado para comenzar la ejecución de código desde una dirección de memoria predeterminada que corresponde a una memoria no volátil (ROM, EPROM, etc) que contiene el programa inicial de carga de un SO.

PC BIOS

En el caso de la IBM-PC, basada en x86, el código (conjunto de funciones o *firmware*) en la ROM se conoce como *Basic Input-Output System (BIOS)*. Las PCs modernas contienen un firmware evolucionado conocido como **UEFI**.

BIOS simplemente carga el contenido del primer bloque de 512 bytes (*master boot record* o *MBR*) del disco primario en la dirección de RAM física `0x7c00` y *salta* (`jmp`) a esa dirección.

El programa cargado deberá cargar en memoria el kernel o un *second stage boot loader* como **GRUB**. GRUB soporta el uso de múltiples OS (Linux, MS-Windows, etc) en diferentes discos o particiones.

El MBR se *graba* durante la instalación o actualización del sistema.

UEFI

UEFI simplifica el proceso de inicio mediante una partición en el disco (con rótulo *EFI*) que contiene archivos de configuración y ejecutables. Un ejecutable puede ser un *boot loader* (como GRUB) o directamente un kernel con soporte UEFI (contiene una cabecera específica).

El firmware carga el archivo de configuración y presenta un menú de opciones de arranque y carga el ejecutable correspondiente.

UEFI también soporta *booteo seguro* que se basa en que un kernel esté *firmado* por el fabricante. En éste caso, las claves privadas de cada fabricante deben almacenarse en la memoria *EPROM*. Esto complica las actualizaciones frecuentes, práctica común en usuarios Linux.

Otros tipos de *firmware*

En algunas arquitecturas como aquellas basadas en *system on chip (SoC)*, como ARM o RISC-V, comúnmente están configuradas para cargar el kernel desde algún punto específico de algún dispositivo de almacenamiento, comúnmente una tarjeta de memoria de estado sólido (SD cards).

En éstos sistemas el kernel se carga a partir de una dirección física de memoria predefinida y especificada para cada *board*.

Inicio del SO

El kernel debe estar compilado y enlazado a partir de la dirección de carga. En su inicio realiza los siguientes pasos:

1. Inicializa subsistemas.
2. Lanza el primer proceso de usuario (`init` en sistemas tipo UNIX).
3. Este proceso ejecuta un programa de control de la terminal (`tty`).
4. `tty` lanza `login` .
5. `login` lanza el intérprete de comandos o `shell`.